Voice Appointment Agent

A Deep Dive: how a phone call becomes a booked appointment

Portfolio explainer, built from the repository source

The one-sentence version

Somebody phones a small business. Twilio answers the call and turns the caller's speech into text. That text is handed to Google's Gemini 2.5 Flash language model, which works out what the caller actually wants (book, reschedule, or cancel) and pulls out the details (name, date, time). A small database then really does book the appointment, refusing to double-book. The whole thing is designed so that the expensive parts (the phone line and the language model) can be unplugged and replaced with free stand-ins, which is why it can be tested 54 times over without spending money.

Contents

1 What this project is, and what it is not

1.1 The problem it addresses

A clinic, a salon, or a repair shop loses bookings when nobody picks up the phone. The caller does not leave a voicemail; the caller phones the next business on the list. The traditional fix is a "phone tree" ("press 1 for bookings"), which callers dislike and which cannot handle a sentence like "can you move my Tuesday thing to Thursday morning?".

This project is a working demonstration of the modern alternative: a real telephone number that a real person can speak to in ordinary sentences, backed by a language model that understands the request, and by a calendar that genuinely reserves the slot.

1.2 Scope, stated honestly and up front

This is a demonstration of a pattern, not a deployed product

Read the repository carefully and the boundary is clear, and the project's own README is unusually candid about it. What is genuinely proven: the conversation logic, the calendar, the conflict detection, the HTTP endpoints, and that the real Gemini model classifies clean typed English correctly. What is *not* proven: that any of this survives contact with a real human voice on a real phone line. Twilio's speech-to-text output is noisy (mumbling, background noise, half-sentences), and the language model in this repository has only ever been shown clean, hand-typed sentences. That gap, between "the API call works" and "this works on a phone call", is real and the repository says so itself.

Jargon: Telephony

The technology of telephone calls. Here it means the machinery that makes a physical phone ring and carries the audio, which this project rents from a company called Twilio rather than building itself.

1.3 Size and shape of the codebase

Roughly 1,900 lines of Python across nine source modules, five test files, and a 25-scenario evaluation harness. It is a compact, focused project. Every tracked file was read in full to build this document.

```
voice-agent
├── voice_agent/     the application itself
│   ├── app.py     FastAPI web service Twilio talks to
│   ├── conversation.py     the state machine: all the business logic
│   ├── intent_engine.py     the Gemini interface + implementation
│   ├── calendar_store.py     SQLite + Google Calendar backends
│   ├── session_store.py     per-call memory and transcript
│   ├── twiml.py     builds Twilio's XML instructions
│   ├── config.py     environment variables, business rules
│   └── cli.py     places a real outbound test call
├── eval/     the offline evaluation harness
│   ├── scenarios.py     the 25 scripted conversations
│   ├── scripted_engine.py     free stand-in for Gemini
│   └── run_eval.py     prints the pass/fail report
├── tests/     54 pytest tests
│   ├── test_calendar_store.py     13 tests, no network
│   ├── test_twiml_endpoints.py     7 tests via a fake HTTP client
│   ├── test_eval_scenarios.py     runs all 25 scenarios
│   ├── test_llm_intent.py     6 REAL, billed Gemini calls
│   └── test_twilio_integration.py     signature check + 1 free lookup
├── .env.example     placeholders only; real secrets live outside
├── requirements.txt
└── README.md
```

Figure 1: The repository. Note the deliberate separation: `voice_agent/` is production code, `eval/` contains a test double that must never run in production.

2 The concepts you need first

This section defines everything the rest of the document assumes. If a term is already familiar, skip it.

Jargon: API (Application Programming Interface)

An agreed way for one program to ask another program to do something. When this project "calls the Gemini API", it sends a message over the internet to Google's servers and gets a reply back. Google charges a small amount per call.

Jargon: Webhook

The reverse of the usual arrangement. Normally your program calls someone else's server. With a webhook, you give *them* a web address, and *they* call *you* when something happens. Twilio does exactly this: when a call comes in, Twilio makes an HTTP request to this project's web address and asks "what should I say?". This is why the project is a web server at all.

Jargon: HTTP request / POST

The message format the web runs on. A "POST" request is one that carries a body of data, as opposed to just asking for a page. Twilio POSTs the details of the call (who is calling, what they said) to this service.

Jargon: TwiML (Twilio Markup Language)

Twilio's instruction format: a small XML document telling Twilio what to do next on the call. `<Say>` means speak this text aloud. `<Gather>` means listen to the caller and send me what you heard. `<Hangup>` means end the call. This project's entire job, from Twilio's point of view, is to answer with TwiML.

Jargon: XML

A text format that stores structured data using angle-bracket tags, like `<Say>Hello</Say>`. TwiML is one specific flavour of it.

Jargon: LLM (Large Language Model)

A program trained on enormous quantities of text that can read a sentence and produce a sensible reply. Gemini 2.5 Flash is Google's fast, cheap model. "Flash" signals that it is tuned for speed over maximum capability, which matters here because a caller on a phone will not tolerate long silences.

Jargon: Slot filling

The task of collecting a fixed set of facts through conversation. The "slots" here are name, phone, date, time, and duration. The agent's job each turn is to fill whichever slots are still empty, one question at a time, until it has enough to book.

Jargon: State machine

A program that is always in exactly one of a limited number of situations ("states"), and moves between them according to fixed rules when something happens. A conversation is a natural fit: "no idea what they want yet" becomes "booking, still need a name" becomes "booking, waiting for them to confirm".

Jargon: SQLite

A complete database that lives in a single ordinary file on disk, with no separate server process to install or run. Ideal for a small project. Its main limitation, which matters later, is that only one thing may write to it at a time.

Jargon: Interface (in programming)

A published list of operations that something promises to provide, without saying how. If you write your program against the interface "a calendar can book, cancel and reschedule", you can later swap the real calendar for a fake one and your program cannot tell. This single idea is the backbone of this project's architecture, and Section 5 is devoted to it.

Jargon: Test double

A stand-in used during testing in place of the real, slow, or expensive thing, the way a stunt double stands in for an actor. This project has one: a fake "language model" that is really just pattern-matching rules.

Jargon: E.164

The international standard for writing phone numbers unambiguously: a plus sign, a country code, then the number, with no spaces or dashes. `+15551234567`. Remember this one; an inconsistency around it is a real bug found in Section 9.

3 Architecture: the shape of the whole system

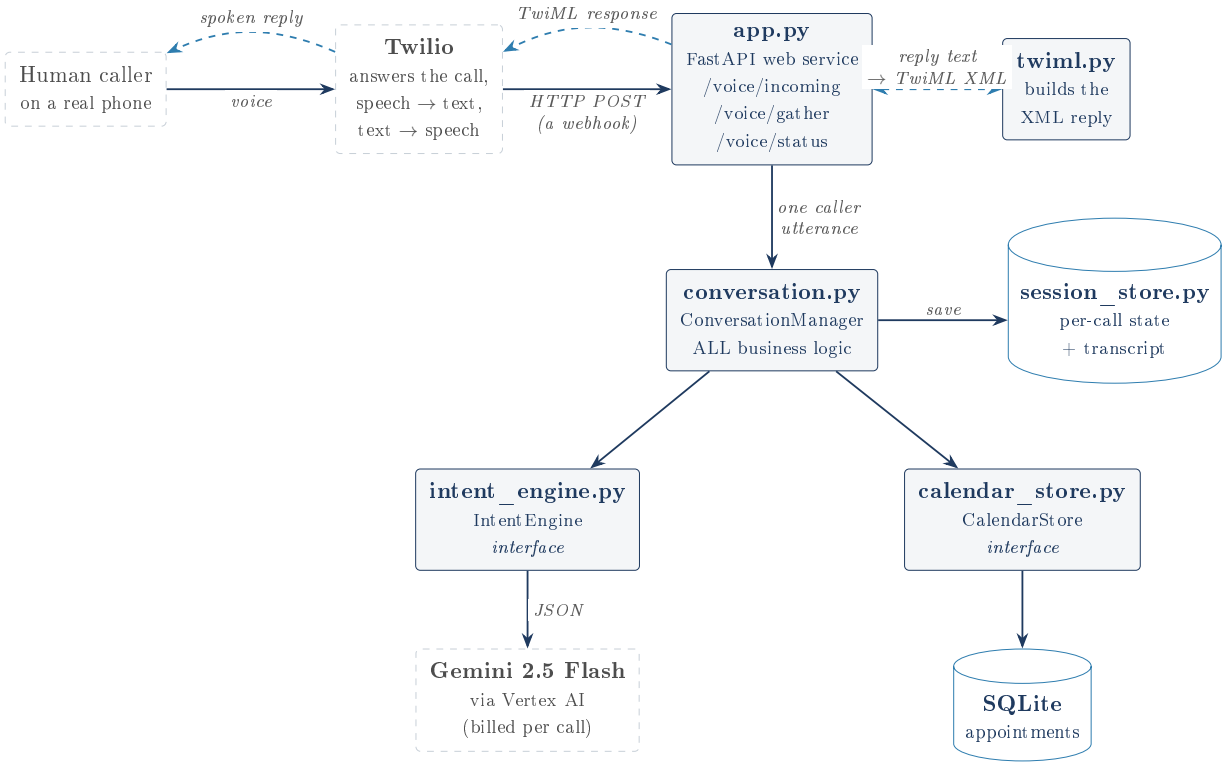

Figure 2: The whole system. Solid arrows are requests going in; dashed arrows are responses coming back. The two boxes marked *interface* are the load-bearing design decision: everything below them can be swapped out.

3.1 The single most important structural fact

`conversation.py` contains all the business logic, and it does not know that Twilio or Gemini exist. It talks only to the two interfaces. Read its import list and this is provable: it imports `IntentEngine` and `CalendarStore`, never `GeminiIntentEngine` or any Twilio module.

That is what makes the rest of the project possible, and it is the thing to be able to explain under questioning. It is unpacked properly in Section 5.

4 The call flow: what actually happens, in order

The counter-intuitive part of telephony webhooks is that a single phone call is not one continuous conversation from the server's point of view. It is a series of *separate, unrelated HTTP requests*. The server has no memory between them. This drives the whole design of `session_store.py`.

Why the server needs a memory at all

Each time the caller speaks, Twilio makes a brand new HTTP request. As far as the web service is concerned, that request could be from anyone, about anything. Everything the agent knows about this call, the name collected two turns ago, the date, whether they confirmed, must therefore be written to a database and read back on every single turn. The key used to look it back up is Twilio's `CallSid`, a unique identifier Twilio assigns to each call and includes in every request.

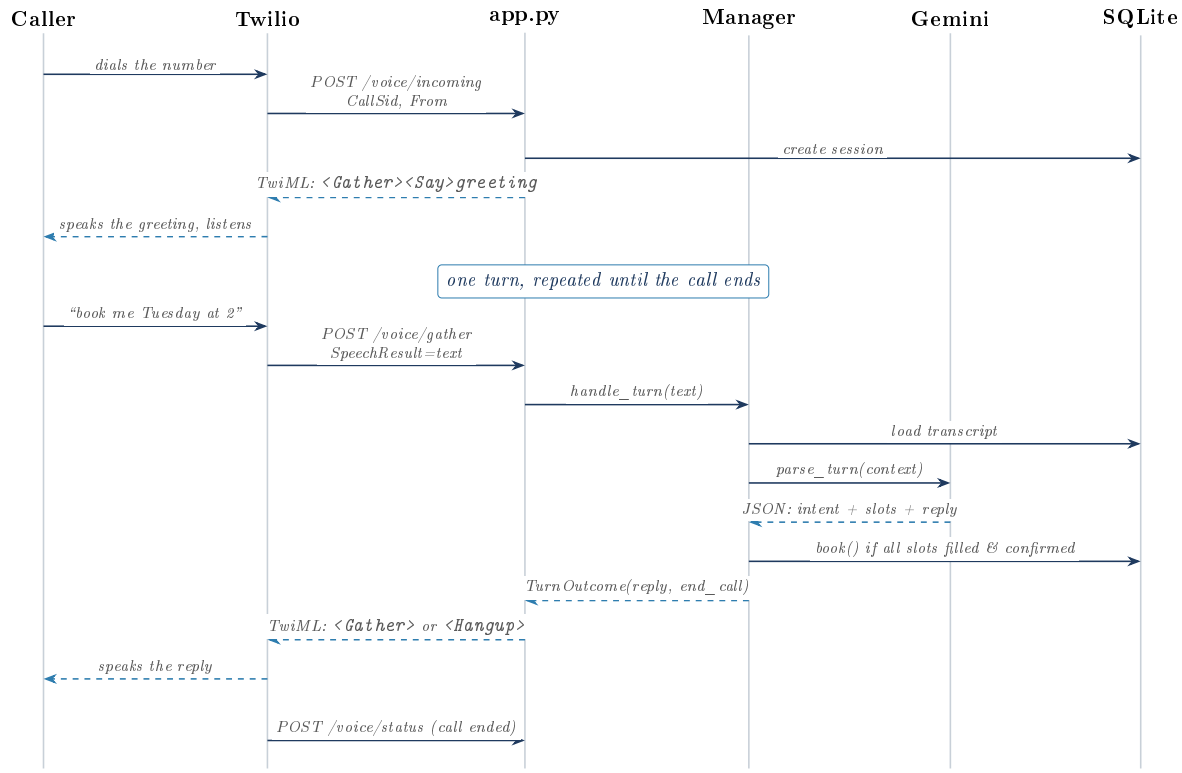

Figure 3: One call, turn by turn. Every horizontal arrow from Twilio to `app.py` is a *separate HTTP request* with no memory of the last one, which is why the transcript is re-loaded from SQLite on every turn.

4.1 The silence problem, and the fix

This is one of the genuinely non-obvious engineering details in the project, and it is worth understanding because it is the kind of thing that only surfaces when you read the vendor documentation properly.

<Gather> does not call you back when the caller says nothing

By default, Twilio's `<Gather>` requests its `action` URL only when it actually captures speech. If the caller is silent, Twilio simply gives up and moves on to the *next* instruction in the TwiML document instead. The consequence is severe: all the careful "sorry, could you repeat that?" retry logic in `conversation.py` would be unreachable for a silent caller, which is precisely the caller who most needs it.
The fix is one attribute, `action_on_empty_result=True`, set on every `<Gather>` in `voice_agent/twiml.py`. With it, silence also posts back to `/voice/gather`, just with an empty `SpeechResult`, so every possible outcome flows through the one state machine.

`twiml.py` is only 40 lines and builds exactly two documents:

```python
def build_gather_twiml(say_text: str, gather_action_url: str) -> str:
    response = VoiceResponse()
    gather = Gather(
        input="speech",
        action=gather_action_url,
        method="POST",
        speech_timeout="auto",
        timeout=GATHER_TIMEOUT_SECONDS,      # 6 seconds
        action_on_empty_result=True,          # <- the fix described above
    )
    gather.say(say_text)
    response.append(gather)
    # Belt and braces: only reached if Twilio somehow does not re-request
    # the action URL (e.g. the call already ended).
    response.say("We didn't receive any input. Goodbye.")
    response.hangup()
    return str(response)
```

Listing 1: voice_agent/twiml.py, the essential part

The trailing `<Say>`/`<Hangup>` pair is a deliberate safety net. If Twilio ever does fall through, the call ends politely rather than hanging in silence.

5 The seams: why two interfaces change everything

5.1 The problem being solved

The evaluation harness runs 25 scenarios. Several are five turns or more. Run those against the real Gemini model and each run is over 100 billed API calls, takes real time, and, because language models are not perfectly deterministic, might give a different answer tomorrow. Doing that on every code change is unaffordable and unreliable.

Jargon: Deterministic

Always producing exactly the same output for the same input. Ordinary code is deterministic; a language model largely is not, which is why a test that depends on one is fragile.

5.2 The solution

Define what the conversation manager *needs* rather than what it will *use*:

```python
class IntentEngine(abc.ABC):
    @abc.abstractmethod
    def parse_turn(self, ctx: TurnContext) -> TurnResult: ...

class CalendarStore(abc.ABC):
```

```
    @abc.abstractmethod
    def find_conflict(self, start, duration_minutes, exclude_id=None): ...
    @abc.abstractmethod
    def book(self, name, phone, start, duration_minutes) -> Appointment: ...
    @abc.abstractmethod
    def get(self, appointment_id) -> Appointment: ...
    @abc.abstractmethod
    def list_by_phone(self, phone, active_only=True) -> list[Appointment]: ...
    @abc.abstractmethod
    def reschedule(self, appointment_id, new_start, new_duration_minutes=None): ...
    @abc.abstractmethod
    def cancel(self, appointment_id) -> Appointment: ...
```

Listing 2: The two interfaces, reduced to their essence

Jargon: Abstract base class / `abc.ABC`

Python's way of writing an interface. Marking a method `@abstractmethod` means "every real implementation must provide this"; Python refuses to create an object from a class that has not filled them all in. It turns a design intention into an error the language enforces.

Now the identical `ConversationManager` runs in three different worlds:

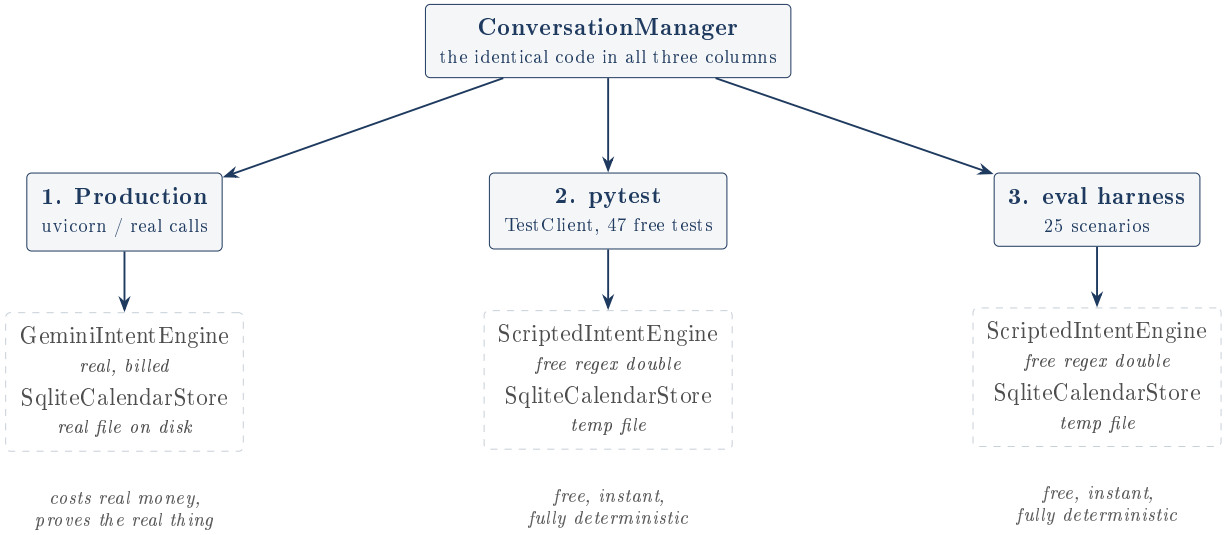

Figure 4: One state machine, three wirings. This is dependency injection: the manager is *handed* its collaborators rather than creating them, so the caller decides whether they are real or fake.

Jargon: Dependency injection

A long name for a simple habit. Instead of a class building the things it needs (`self.engine = GeminiIntentEngine()`, which welds it to Gemini forever), it accepts them as arguments (`def __init__(self, intent_engine, ...)`). Whoever creates the object chooses what to pass. That choice point is the "seam" that testing needs.

The test double is honest about being one, and that matters

It would be easy, and dishonest, to present `ScriptedIntentEngine` as a "fallback" so the project could claim it works without an API key. The repository refuses to do this. Its docstring states plainly: *"This is a TEST DOUBLE, not a production fallback."* It lives in `eval/`, not in `voice_agent/`, so it is physically outside the production package. `app.py` always wires up the real Gemini engine unless a test explicitly passes something else. The "LLM-driven" claim

therefore survives scrutiny: the shipped path really does use the model.

6 Module by module

6.1 voice_agent/config.py

The smallest module, and quietly disciplined. Every value is read from the environment *at call time*, never cached at import time, and never logged.

```python
def _require(name: str) -> str:
    value = os.environ.get(name, "").strip()
    if not value:
        raise ConfigError(f"Required environment variable {name} is not set")
    return value

BUSINESS_HOURS_START = 9    # 9:00 local
BUSINESS_HOURS_END = 17     # 17:00 local
DEFAULT_APPOINTMENT_DURATION_MINUTES = 30
MAX_RETRY_COUNT = 3         # consecutive unclear turns before handoff
MAX_TURN_COUNT = 14         # total turns before forced handoff
```

Listing 3: The required-variable helper and the business rules

Two small decisions worth noticing. Reading at call time rather than import time is what lets a test set an environment variable and have it actually take effect. And failing loudly with `ConfigError` when a credential is missing is better than starting up and failing mysteriously on the first real call.

Jargon: Environment variable

A named value supplied to a program by the system that starts it, rather than written in the code. It is the standard way to supply passwords and API keys, precisely so they never appear in the source code and never get committed to version control.

6.2 voice_agent/session_store.py

Two SQLite tables, and the answer to "how does a stateless web service remember a phone call?".

- `sessions`: one row per call, keyed by `call_sid`. The entire `SessionState` object is serialised to JSON and stuffed into one `state_json` column.
- `transcript`: one row per utterance, for auditing what was actually said.

Jargon: JSON, and serialisation

JSON is a plain-text format for structured data. *Serialising* means flattening a live object into text so it can be stored; *deserialising* is rebuilding the object from that text. Here, the whole session state becomes one string in one column.

Why store the state as one JSON blob instead of proper columns?

It is a real trade-off, chosen deliberately. Storing JSON means adding a new field to `SessionState` requires no database migration; the code just works. The price is that you cannot query on those fields ("show me all calls stuck on the phone slot" is not expressible in SQL here), and nothing validates the shape. For a single-purpose store that is only ever read back whole, by primary key, this is a sound trade. It would not be for a table you needed to report on.

The `save` method uses an *upsert*, which inserts a row or updates it if the key already exists, in one atomic statement:

```
INSERT INTO sessions (call_sid, state_json, created_at, updated_at)
VALUES (?, ?, ?, ?)
ON CONFLICT(call_sid) DO UPDATE
  SET state_json = excluded.state_json, updated_at = excluded.updated_at
```

Listing 4: The upsert in session_store.save

Jargon: Parameterised query (the ? marks)

The `?` placeholders are not string formatting. The database receives the query and the values separately, so a value can never be mistaken for SQL code. This is the correct defence against *SQL injection*, where hostile input like `'; DROP TABLE appointments; --` would otherwise be executed as a command. Every query in this project is parameterised, which is exactly right, and worth pointing out because a caller's spoken name goes into the database.

`mark_abandoned_if_unresolved` is the neat part. When Twilio reports the call ended, this checks whether any outcome was ever reached; if not, the call is recorded as `abandoned`. Crucially it refuses to overwrite an outcome that already exists, so hanging up after a successful booking still counts as booked. There is a test for exactly this.

6.3 voice_agent/calendar_store.py

The largest module (339 lines) and the one that does the real work. It defines the `CalendarStore` interface, two implementations, and the conflict rule.

6.3.1 The conflict rule

Everything hinges on eight characters:

```
def _overlaps(a_start, a_end, b_start, b_end) -> bool:
    """Half-open interval overlap: touching endpoints (a_end == b_start) is NOT a conflict."""
    return a_start < b_end and b_start < a_end
```

Listing 5: The half-open interval overlap test

Why < and not <=

Two appointments where one ends at exactly 9:30 and the next starts at exactly 9:30 do not overlap; they are back to back, which is what a busy clinic wants. Using `<=` would report a false conflict and refuse a perfectly good booking. This is the classic *half-open interval*: the start instant belongs to the appointment, the end instant does not.

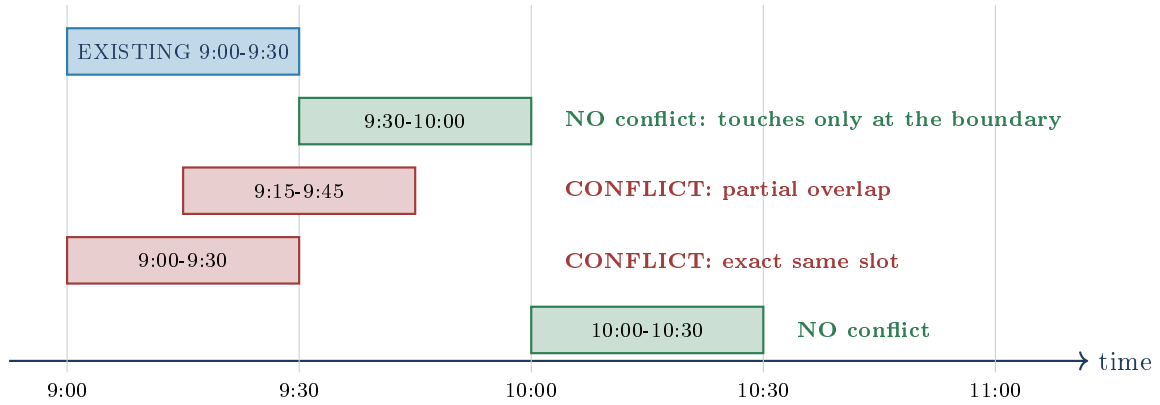

Figure 5: `a_start < b_end and b_start < a_end`, drawn. The green 9:30 case is the one a naive `<=` implementation gets wrong, and it is covered by scenario `boundary_adjacent_appointments_no_conflict`.

6.3.2 `SqliteCalendarStore`, the default

One table, `appointments`, with `status` either `booked` or `cancelled`. Cancelling is a *soft delete*: the row stays, the status changes, so the history survives and the conflict check simply ignores cancelled rows.

Jargon: Soft delete

Marking a record as inactive rather than physically removing it. You keep an audit trail and can undo; the cost is that every query must remember to filter out the dead rows.

`find_conflict` reads the entire table into Python on every booking

The query is `SELECT * FROM appointments WHERE status = 'booked'` with *no time range filter at all*. Every booked appointment ever made, past and future, is loaded and looped over in Python to find an overlap. There is also no index on `start_time` or `phone`.
At this project's scale (one phone line, a handful of appointments) this is completely invisible, and choosing simplicity over premature optimisation is defensible. But it is *O(n)* work per booking that a database is built to do in *O(log n)*, and the fix is a one-line `WHERE start_time < ? AND ...` clause plus an index. Worth being able to say "I know, here is exactly why it does not matter yet, and here is the fix" rather than being shown it.

Jargon: O(n) and O(log n)

Shorthand for how work grows with data size. *O(n)*: ten times the appointments means ten times the work. *O(log n)*: ten times the appointments means only slightly more work, because an index lets the database skip most rows. This is what indexes are for.

6.3.3 `GoogleCalendarStore`, the optional adapter

A complete second implementation against the real Google Calendar API v3, using `events.insert`, `events.patch`, `events.delete` and `freebusy.query` for conflict detection. `get_calendar_store()` selects it automatically when the environment variable `GOOGLE_CALENDAR_CREDENTIALS_PATH` is set and the file it names exists; otherwise the SQLite store is used.

Jargon: Adapter (design pattern)

Code whose only job is to make one thing present itself with the shape of another. Here it makes Google Calendar look like a `CalendarStore`, translating between appointment ids and Google event ids so nothing upstream has to know the difference.

The Google client libraries are imported *inside* `__init__`, not at the top of the file. That is deliberate: the default SQLite install then does not need them present.

The Google adapter has never been run, and it has at least one real defect

The README is upfront that no Google Calendar credentials were ever provided, so this class has never executed against a live calendar. It is not a stub, but it is unverified. Reading it closely, two concrete problems are visible without running it:

One. `cancel()` calls `events().delete()`, which destroys the event outright, then *fabricates* a return value with `start_time=datetime.now()`. But `conversation.py` uses that return value to speak to the caller: `f"Done, your appointment for {when} is cancelled"`. On the Google backend the agent would therefore read back *the current time* rather than the appointment's real time. On SQLite this is invisible, because there `cancel()` returns the true row.

Two. The `Appointment` dataclass declares `id: int`, but Google event ids are strings, and this class assigns them straight into that field. Python does not enforce type hints at runtime so nothing crashes, but the declared contract is being broken, and `SessionState.target_appointment_id` is likewise typed `Optional[int]`.

Neither is hard to fix. Both are the predictable cost of writing an implementation you cannot run, which is itself the honest lesson here.

6.4 voice_agent/intent_engine.py

This is where the language model lives. `TurnContext` in, `TurnResult` out.

```python
@dataclass
class TurnContext:
    caller_number: str
    utterance: str                      # "" if <Gather> captured no speech
    history: list[dict]                 # [{"speaker": "agent"/"caller", "text": ...}]
    known_slots: dict                   # what we have collected so far
    current_action: Optional[str]       # book/reschedule/cancel, if established
    today: date                         # injected, never datetime.now()

@dataclass
class TurnResult:
    reply: str                          # what to say back
    intent: str                         # book|reschedule|cancel|unclear|goodbye
    name: Optional[str] = None
    phone: Optional[str] = None
    date_iso: Optional[str] = None      # "YYYY-MM-DD"
    time_24h: Optional[str] = None      # "HH:MM"
    duration_minutes: Optional[int] = None
    confirmed: Optional[bool] = None
```

Listing 6: The data crossing the boundary

`confirmed` is three-valued, and that is deliberate

`Optional[bool]` means it can be `True`, `False`, or `None`, and all three are distinct: "yes, book it", "no, that's wrong", and "I have not been asked yet". A plain `True/False` would silently merge "said no" with "never asked", and the agent would either book without consent or never book

at all. The same three-valued logic is why the code tests `if slots.get("confirmed") is not True` rather than `if not confirmed`.

6.4.1 Making the model fast enough for a phone call

Three settings do the work, and the reasoning behind them is the most interesting engineering judgement in the project:

```
config=types.GenerateContentConfig(
    system_instruction=system_prompt,
    thinking_config=types.ThinkingConfig(thinking_budget=0),  # turn thinking OFF
    response_mime_type="application/json",                    # must be JSON
    response_schema=self._response_schema(),                  # this exact shape
    temperature=0.1,                                          # near-deterministic
)
```

Listing 7: GeminiIntentEngine.parse_turn, the configuration

Sometimes engineering means turning the clever feature off

Gemini 2.5 Flash "thinks" before answering by default, which improves hard reasoning and costs seconds and tokens. The README records an actual measurement: an exploratory call spent 56 thinking tokens on a one-word answer. On a phone call, multiple seconds of dead air is a product failure; the caller assumes the line dropped. So thinking is set to zero. The measured result with thinking off plus a strict schema was 1.8 seconds and a clean structured reply.
This is a genuinely good instinct to be able to articulate: the right configuration depends entirely on the product, and for a live phone turn, latency beats depth.

Jargon: Token

The unit language models read, write, and bill in: roughly a short word or word-fragment. "Thinking tokens" are ones the model generates privately to reason before answering. You pay for them, and you wait for them.

Jargon: Temperature

A dial for randomness. At 0 the model gives its single most likely answer every time; higher values invite variety. For creative writing you want it high. For extracting a date from a sentence you want it near zero, hence 0.1.

Jargon: Response schema, and structured output

Rather than asking the model in prose to "please reply in JSON" and hoping, the schema is supplied to the API as a formal specification and the platform constrains the output to match it. The difference matters: politely asking sometimes yields "Sure! Here's the JSON: {...}", which then fails to parse. This is the robust way to get machine-readable output.

6.4.2 Defensive handling of the reply

The parsing is properly paranoid, which is correct when the input comes from a model:

```
intent = data.get("intent", "unclear")
if intent not in VALID_INTENTS:
    intent = "unclear"           # an unknown intent degrades to "ask again"
...
```

```
reply=data.get("reply") or "Sorry, could you say that again?",
name=data.get("name") or None,   # "" and null both collapse to None
```

Listing 8: Never trust the model's output shape

Any exception at all from the API call becomes an `IntentEngineError`, which `conversation.py` turns into a graceful spoken handoff rather than a crash. An unknown intent degrades to `unclear`, which routes into the retry path. Both are the right failure direction: a caller hears a sentence, never silence.

6.5 `voice_agent/conversation.py`: the state machine

The heart of the project. 338 lines, no Twilio, no Gemini, no SQL.

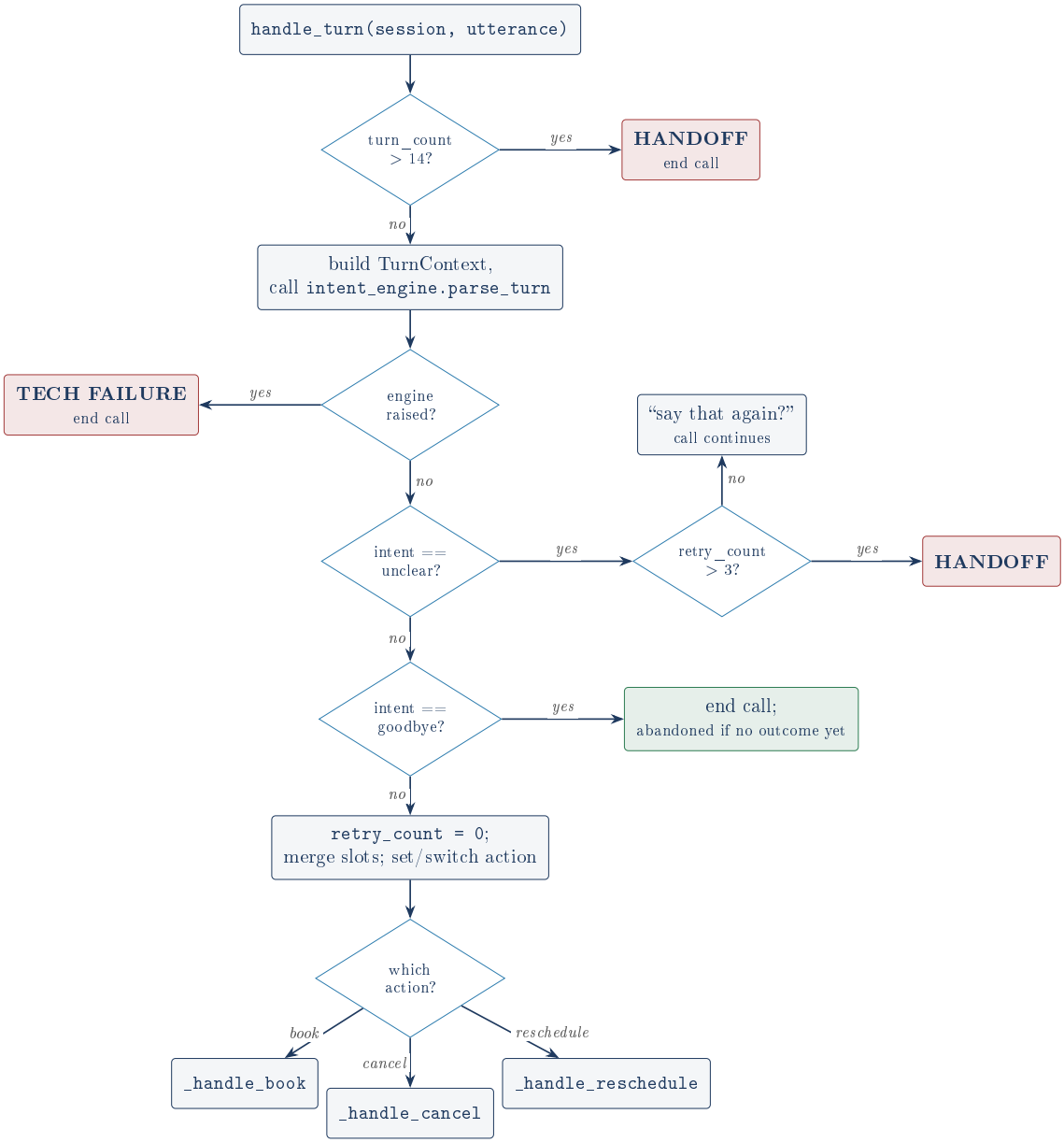

Figure 6: `handle_turn`, the top-level decision flow. Note the order: the safety caps are checked *before* anything expensive happens, so a runaway call cannot keep billing Gemini.

6.5.1 The two safety nets, and why there are two

A retry cap and a turn cap catch different failures

`MAX_RETRY_COUNT = 3` counts *consecutive unintelligible* turns. It catches the bad line, the caller in a tunnel, the background noise.
`MAX_TURN_COUNT = 14` counts *all* turns, and catches the case the retry cap cannot: a caller whose every turn is perfectly intelligible but never actually completes anything. Say "sometime in the morning" forever and `retry_count` resets to zero every single turn, because each turn was a valid continuation. Without the second cap, the call never ends. Scenario 24 tests exactly this, and asserts `retry_count == 0` to prove it was the turn cap and not the retry cap that fired.
That is a genuinely thoughtful pair of tests: the assertion proves *which* mechanism saved the call, not merely that something did.

Note also `session.retry_count = 0` is reset on any clear turn, so the cap means "three in a row", not "three per call". That is the humane choice.

6.5.2 The booking path, traced

`_handle_book` runs a strict gauntlet, in this order:

_handle_book, in order

```
1. missing = any of (name, phone, date_iso, time_24h) still empty?
   if so:  return the model's question, keep the call open
2. slots["confirmed"] is not True?
   if so:  return the model's read-back, keep the call open
3. parse date+time into a real datetime
   unparseable:  clear time, clear confirmed, re-ask
4. _validate_business_slot(dt)
   in the past, or outside 9:00-17:00:  clear time, clear confirmed, re-ask
5. calendar_store.book(...)
   ConflictError:  clear time, clear confirmed, offer to pick another time
6. success:  outcome = "booked", speak the confirmation
```

The clear-time-and-confirmation reset is the subtle bit

Every rejection path does the same two things: `slots["time_24h"] = None` and `slots["confirmed"] = None`. Miss either and the bug is nasty. Leave the time set and the next turn re-validates the same bad time forever. Leave `confirmed` at `True` and the caller's "yes" to the *rejected* 7am silently becomes a "yes" to whatever time they mention next, booking without real consent. Resetting both forces a fresh, explicit confirmation of the corrected time. This pattern is repeated identically in `_handle_reschedule`.

6.5.3 Validation, and the injectable clock

```python
def __init__(self, intent_engine, calendar_store, session_store,
             now_fn: Callable[[], datetime] = datetime.now):   # <- injected clock
    ...

def _validate_business_slot(self, dt):
    now = self.now_fn()                        # not datetime.now()
    if dt <= now:
        return "That date and time has already passed. ..."
    if not (config.BUSINESS_HOURS_START <= dt.hour < config.BUSINESS_HOURS_END):
        return "That's outside our business hours ..."
```

```
    return None
```

Listing 9: Why now_fn exists

Injecting the clock is what makes the tests possible at all

A test asserting "the appointment lands on 2026-07-15 at 14:00" after the caller says "July 15th at 2pm" is only correct on days when that date is still in the future. Call `datetime.now()` directly and the test suite quietly rots: it passes today and starts failing on 16 July 2026 for reasons unrelated to any code change. Threading a `now_fn` through and pinning every scenario to `FIXED_NOW = datetime(2026, 7, 6, 13, 0)` makes time itself an input. Notice `FIXED_NOW` is at 13:00: mid-afternoon, deliberately, so a same-day "past" time (10am) can be tested without also tripping the business-hours check, keeping the two rules independently testable. That is careful test design.

Note the business-hours check tests `dt.hour` only, so 16:45 passes even though a 30 minute appointment then runs 15 minutes past closing. `config.py` concedes this in a comment ("last bookable start is 16:xx for a 30 min slot"). It is a known simplification, not an oversight.

6.5.4 Changing your mind mid-call

```
if session.action is None:
    session.action = result.intent
elif session.action != result.intent:
    # Caller changed their mind mid-call: start this new action clean.
    session.action = result.intent
    session.slots = {}
    session.target_appointment_id = None
```

Listing 10: The action switch

Wiping the slots on a switch is right: a date gathered for a booking should not leak into a cancellation. The cost is that a caller who switches and switches back re-answers everything. For a phone agent, correctness beats convenience here.

6.6 `voice_agent/app.py`: the web layer

Deliberately thin. Its whole job is: receive an HTTP form, call the manager, turn the answer into TwiML.

Jargon: FastAPI

A Python web framework: a library that handles the plumbing of receiving HTTP requests and routing them to your functions, so you write only the logic. The `@app.post("/voice/gather")` line above a function means "run this when a POST arrives at that address".

6.6.1 The app factory, and a clever import trick

`create_app(...)` takes optional stores and an engine. Pass nothing and it wires up the real production dependencies; pass fakes and you get a test instance. But that creates a problem: `uvicorn` needs a module-level `app` object, and building one would demand real credentials, which would make merely *importing* the module fail in tests.

```
def __getattr__(name: str):
    if name == "app":
        return create_app()
```

```
    raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
```

Listing 11: PEP 562 module-level __getattr__

Lazy construction via module __getattr__

This hook runs only when someone actually asks for an attribute the module does not otherwise have. So `import voice_agent.app` costs nothing and needs no credentials, while `uvicorn voice_agent.app:app` touches `app`, triggers the hook, and builds the real thing. It neatly resolves the tension between "uvicorn wants a module-level object" and "tests must import this without credentials". It is niche but legitimate Python (PEP 562), and it is well commented in the source.

6.6.2 Signature verification

Jargon: Webhook signature

Your webhook address is a public URL: anyone who finds it can POST to it and pretend to be Twilio. To prevent this, Twilio signs each request with a cryptographic hash computed from the URL, the form data, and your secret auth token, in the `X-Twilio-Signature` header. You recompute it with the same token; if they match, the request really came from Twilio, because only Twilio and you know the token.

The signature check silently does nothing in the default configuration

This is the most significant finding in the review, and it was confirmed by running the code, not by reading it.

```
if app.state.enforce_signature:
    base_url = config.public_webhook_base_url()
    auth_token = os.environ.get("TWILIO_AUTH_TOKEN", "")
    if base_url and auth_token:              # <- if either is missing, NO CHECK RUNS
        if not verify_twilio_signature(request, form, auth_token, base_url):
            return Response(status_code=403, content="Invalid Twilio signature")
```

`PUBLIC_WEBHOOK_BASE_URL` is blank by default, and `.env.example` explicitly instructs "Leave blank for local development and testing". So with `enforce_signature=True`, the nominally secure setting, an entirely unsigned request is accepted. Driving the real app with a `TestClient` confirms it: base URL unset plus no signature header at all returns **HTTP 200**. Set the base URL and the same unsigned request correctly returns **HTTP 403**, so the check itself is implemented correctly.

There is a real reason for the design: Twilio signs the *public* URL it requested, so without knowing that URL you cannot recompute the signature, and there is genuinely nothing to check against locally. The danger is the failure *direction*. This *fails open*: the consequence of a missing setting is silently no security, not a refusal to start. Deploy this behind a real URL and forget the variable, and the booking endpoints are wide open to anyone who finds them, with no warning in the logs.

The fix is small and worth stating in an interview: if `enforce_signature` is true and the base URL is missing, refuse to start, or at minimum log a loud warning on every request. Make the safe path the default and the insecure path something you must deliberately choose, for example an explicit `allow_unsigned=True`.

6.6.3 Endpoint behaviour

`/voice/status` accepts Twilio's terminal statuses (`completed`, `failed`, `busy`, `no-answer`, `canceled`) and marks the call abandoned if no outcome was reached. It returns `204 No Content`, which is correct: Twilio wants nothing back.

6.7 `voice_agent/cli.py`: the outbound test call

The only code in the repository that spends real money on telephony, and it is fenced off accordingly. In order: validate the number against an E.164 regex; refuse outright without a public URL; load and validate the Twilio credentials; then an interactive `[y/N]` confirmation naming both numbers and warning of charges, skippable only with an explicit `--yes`.

The guardrails are real, and this document respected them

Nothing else in the repository imports or invokes `cli.py`; it runs only when a human types the command. In line with the project's budget rule and the instructions for this review, **no phone call was placed and no billed API test was run** while producing this document. The 7 tests marked `real_api` were deliberately deselected, never executed. Every claim below about them rests on reading the code and the README, and is labelled as such.

7 The evaluation harness

7.1 What it actually is

`eval/scenarios.py` defines 25 `Scenario` objects. Each is one or more simulated calls (a phone number plus a list of caller utterances) and a `check` function asserting on the resulting calendar and session state.

What the 25 scenarios do and do not exercise

This is the claim to pin down precisely, because the honest answer is narrower than "25 scenarios pass" sounds, and the repository states it correctly.

Real, and genuinely exercised: the `ConversationManager` state machine, the real `SqliteCalendarStore` on a real temporary database file, the real `SessionStore`, real conflict detection, real business-hours and past-date validation, real retry and turn caps.

Replaced by a test double: the language model, swapped for the regex-based `ScriptedIntentEngine`.

Not involved at all: Twilio, FastAPI, HTTP, TwiML. Utterances are fed straight into `manager.handle_turn`, deliberately "bypassing Twilio and FastAPI entirely" as the module docstring says.

So "25/25" is a strong statement about the booking *logic*, and no statement whatever about speech recognition or the phone line. The FastAPI and TwiML layers are covered separately, by the 7 endpoint tests. The README does not overstate this.

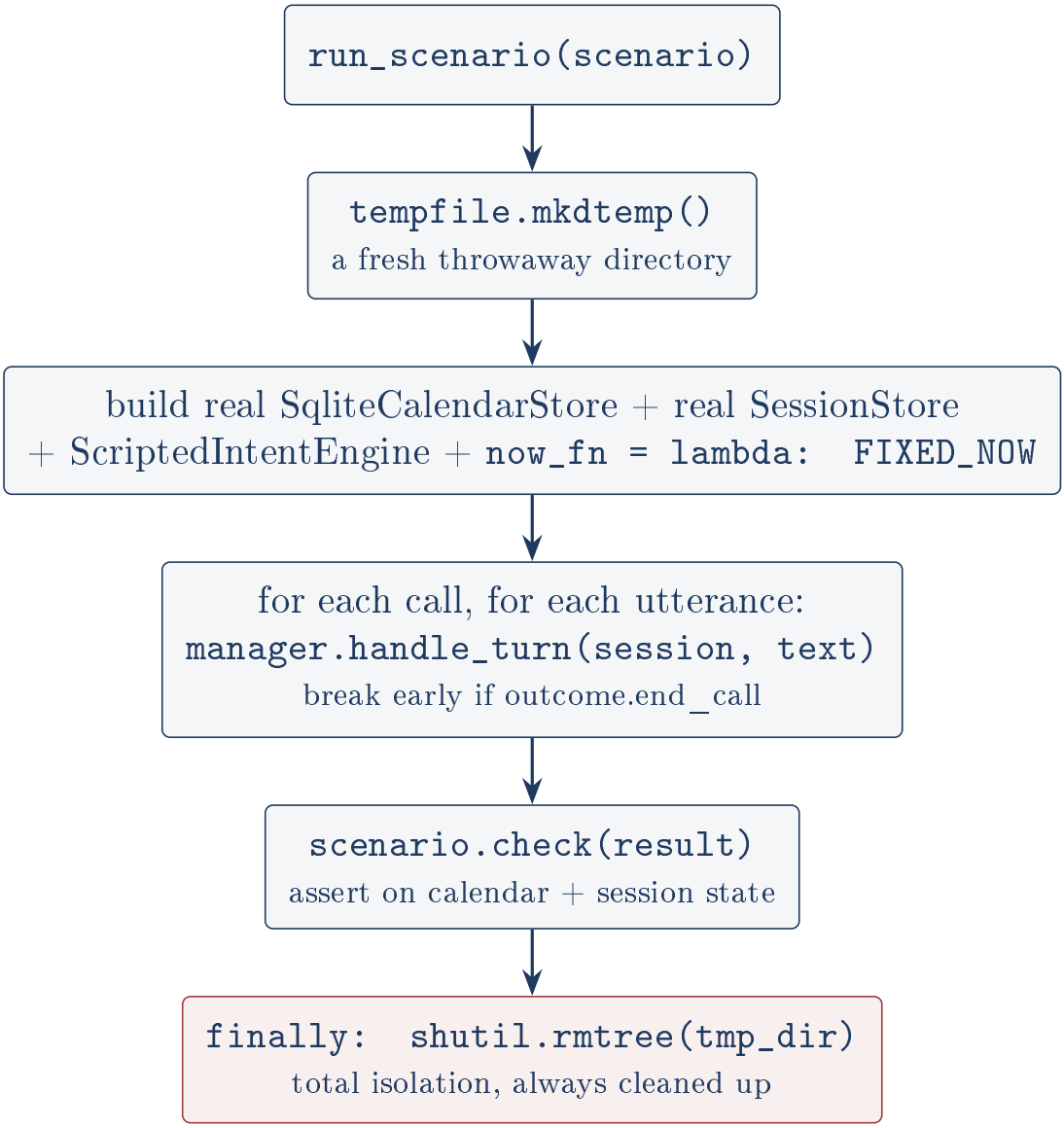

Figure 7: Every scenario gets a brand new temporary database and is destroyed afterwards in a `finally` block, so no scenario can ever influence another.

A nice defensive touch: `scenario.check` is itself wrapped in a `try/except`, so a bug in a check function fails that one scenario rather than crashing the whole run.

7.2 The 25 scenarios, verified

The harness was run for this document. It required no third-party packages and made no network calls, which independently confirms the "runs free" claim.

```
$ python -m eval.run_eval
[PASS] (book) book_simple_one_shot
[PASS] (book) book_multi_turn_slot_filling
... 21 lines omitted for space ...
[PASS] (unclear_speech) max_turn_cap_handoff
[PASS] (conflict) reschedule_conflict_then_alternate

25/25 scenarios passed
```

Listing 12: Observed output, reproducing the committed docs/screenshots/run-output.txt exactly

#	Category	What it actually pins down
1	book	Everything in one sentence, then "yes". The happy path.
2	book	One fact per turn; a spoken phone number wins over caller ID.
3	book	"one hour" overrides the 30 minute default.
4	book	Spoken number stored; caller ID *not* used. See finding below.
5	book	Caller rejects the read-back, corrects, confirms: exactly 1 appointment, not 2.
6	reschedule	Books in call 1, moves it in call 2. State survives across calls.
7	reschedule	Nothing on file: graceful, then goodbye becomes `abandoned`.
8	cancel	Cancel marks status `cancelled`, active list empties, row survives.
9	cancel	Cancel with nothing on file.
10	conflict	Second caller wants a taken slot, is refused, books another.
11	ambiguous	"sometime in the morning" re-asks rather than guessing.
12	ambiguous	"later this week" re-asks; concrete date then accepted.
13	unclear	Silence once, recovers; `retry_count` back to 0.
14	unclear	Four unusable turns: retry cap fires, handoff.
15	abandoned	Twilio status callback marks an incomplete call abandoned.
16	book	7am confirmed, rejected as too early, corrected to 9am.
17	book	9pm confirmed, rejected as too late, corrected to 4pm.
18	book	Today 10am (past, vs `FIXED_NOW` 13:00), rejected, corrected.
19	conflict	**9:00 and 9:30 both book.** The half-open boundary.
20	conflict	10:15 overlaps a 10:00 slot; refused; 11:00 accepted.
21	book	Mid-call switch from book to cancel; slots wiped.
22	book	Two callers, two slots, independent.
23	book	Goodbye after success does *not* overwrite `booked`.
24	unclear	Turn cap fires with `retry_count == 0`.
25	conflict	Reschedule into a taken slot; refused; alternate accepted.

Table 1: The 25 scenarios. 16, 17 and 18 each contain a deliberate extra "yes", because validation only runs after confirmation; without it the scenario would pass without ever executing the code it is named for.

7.3 Do the scenarios have teeth? A verification of the verification

A passing test proves nothing unless it can fail. The README claims the caps were deliberately broken to check the tests noticed. That claim was independently re-tested for this document, by copying the project to a scratch directory and sabotaging it there (the repository itself was never modified):

Sabotage applied	Result	Scenario that caught it
`MAX_RETRY_COUNT` 3 → 999	24/25	`unclear_speech_retry_cap_handoff`
`MAX_TURN_COUNT` 14 → 9999	24/25	`max_turn_cap_handoff`
< → <= in `_overlaps`	24/25	`boundary_adjacent_appointments_no_conflict`

Table 2: Mutation testing, performed for this document. Each sabotage killed exactly the one scenario named for it, and no others. The scenarios are load-bearing, not decorative.

Jargon: Mutation testing

Deliberately introducing a bug to check that the test suite fails. It tests the tests. If you break the code and everything still passes, your tests were measuring nothing. The precision here (each mutation killed exactly one scenario, no collateral) is a sign of well-targeted tests.

7.4 The scripted engine

Roughly a dozen regexes over keywords: `_BOOK_KW`, `_CANCEL_KW`, `_YES_KW`, plus extractors for names, phones, times, dates and durations. Its resolution order matters:

```
if _CANCEL_KW.search(utterance):        new_intent = "cancel"
elif _RESCHEDULE_KW.search(utterance):  new_intent = "reschedule"
elif _BOOK_KW.search(utterance):        new_intent = "book"
elif _GOODBYE_KW.search(utterance):     new_intent = "goodbye"
...
if new_intent is not None:                      intent = new_intent
elif ctx.current_action is not None and (extracted_anything or vague_answer):
    intent = ctx.current_action    # a continuation of what we were doing
else:
    intent = "unclear"             # drives the retry path
```

Listing 13: Intent resolution, in priority order

The `vague_answer` branch is the clever part. "Sometime in the morning" extracts no usable slot, but it is not gibberish either, and conflating the two would make scenario 24 untestable: a real answer that happens to be unhelpful must re-ask *without* incrementing the retry counter.

The scripted engine flatters the state machine, unavoidably

This is worth naming clearly because it bounds what "25/25" means. The scripted engine was written *after* the scenarios, to understand exactly the phrases those scenarios use. Its own docstring admits this: "its natural-language coverage only needs to be as good as the phrases the scenarios use". So the 25 scenarios can never surface a misunderstanding, because the fake model never misunderstands anything; the sentences and the parser were built for each other.

That is the correct trade (the alternative is 100+ billed calls per run, and flaky tests), and the project is open about it. But it means the scenarios test the state machine *given perfect understanding*. Real understanding is tested only by the 6 marked Gemini tests, against clean typed English. Nothing here tests messy speech-to-text output. The right framing under questioning: "25/25 proves the booking logic is correct, and I deliberately made the understanding layer a separate, smaller, honest experiment."

Note too that the parsers are duplicated in spirit: `ScriptedIntentEngine` reimplements relative-date logic ("tomorrow", "next Tuesday") that the Gemini prompt asks the model to do. Two implementations of one rule can drift, though only the Gemini one ships.

8 Testing: what was verified, and how

8.1 The reported numbers, independently reproduced

Every count below was re-verified for this document by installing the pinned requirements into a brand new virtual environment and running the suite. The free suite was run in full; the billed suite was collected but **deliberately never executed**.

Claim in the README	Observed	Verdict
54 tests collected	`54 tests collected in 0.72s`	confirmed
47 run free, 7 billed	`47 passed, 7 deselected`	confirmed
7 marked `real_api`	`7/54 tests collected (47 deselected)`	confirmed, *not run*
25/25 scenarios pass	`25/25 scenarios passed`	confirmed
Installs into a clean venv	installed, exit code 0	confirmed
6 real Gemini calls	6 test functions, 1 call each	*inferred from source*
1.8 s with thinking off	no artefact in the repository	*unverifiable here*
Live tunnel booking	no artefact in the repository	*unverifiable here*

Table 3: Verification of the README's own claims. The free numbers all hold exactly. The last three are the owner's own records of runs that left no committed evidence.

Three claims rest on the owner's word, not on anything in the repository

Stated plainly so they are not defended as if verified:

The 1.8 second latency measurement and the "56 thinking tokens" figure appear only in README prose. No script, log, or artefact reproduces them. They are plausible and specific, but a sceptical interviewer cannot check them, and neither could this review.

The live public-tunnel run (the `localhost.run` tunnel, the genuinely HMAC-signed requests, the row written to the database and then deleted) is the strongest verification claimed, and nothing of it survives in the repository: no log, no transcript, no recording. It was, by design, cleaned up afterwards.

The Gemini cost figure is explicitly labelled in the README as "an estimate from per-token rates, not a verified line-item invoice", which is exactly the right way to say it.

None of this suggests the claims are untrue; the README's overall precision argues strongly the other way, and its willingness to flag its own estimate as an estimate is a good sign. But the distinction between "I ran this and here is the artefact" and "I ran this and cleaned up" is one an interviewer may probe. The cheap fix, for next time, is to commit the log.

8.2 The coverage map

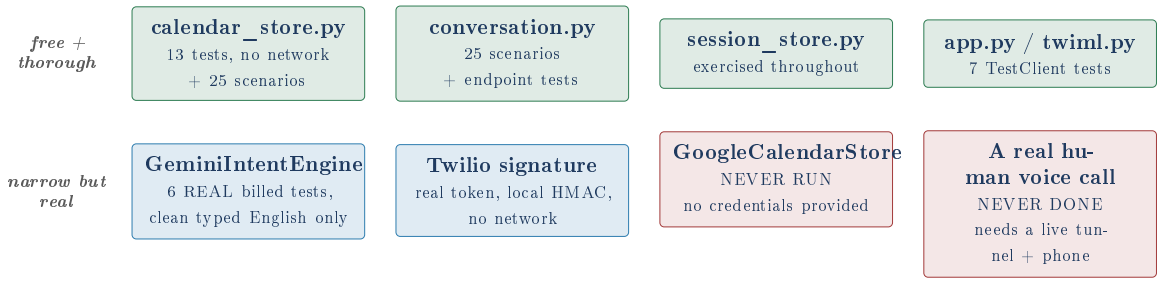

Figure 8: Green: covered free and well. Blue: covered by a small, deliberate, real-API slice. Red: not covered, and the README says so in both cases.

9 Honest findings

Everything below was found by reading the source, and each was confirmed by executing the real code in a scratch copy. None of it appears in the README.

Finding 1: the wrong appointment is cancelled when a caller has two

`_find_target_appointment` takes the first match:

```
matches = self.calendar_store.list_by_phone(phone, active_only=True)
return matches[0] if matches else None
```

and `list_by_phone` sorts `ORDER BY start_time DESC`, meaning *furthest future first*. So a caller with an appointment tomorrow and another next month who says "cancel my appointment" gets **next month's cancelled**, and tomorrow's silently retained. Verified directly: booking 2026-08-01 and 2026-09-01 under one number and asking for the target returns the September one.

The agent does read the date back ("I found your appointment for Tuesday September 1st, should I cancel it?"), so an attentive caller can say no. But the default is wrong: "my appointment" in ordinary speech means the next one. No scenario covers a caller with two active appointments, which is exactly why the suite is green. The fix is `ASC` ordering plus a filter to future appointments; the deeper fix is asking which one.

Finding 2: phone numbers are stored in two incompatible formats

Twilio supplies caller ID in E.164 (`+15551112222`). The scripted engine strips speech to bare digits (`5551112222`). `conversation.py` fills the slot from whichever it has:

```
if not session.slots.get("phone"):
    session.slots["phone"] = session.caller_number    # E.164 from Twilio
```

and `list_by_phone` does an exact string match. So an appointment booked after the caller *spoke* their number is stored under `5551112222` and is **unfindable** when that same person phones back, because lookup then uses `+15551112222`. Verified: booking under the spoken form and looking up the E.164 form returns 0 matches. Cancelling by phone would fail for exactly the caller who was most helpful.

The sharpest part: scenario 4 (`book_explicit_phone_override`) *asserts this behaviour as correct*, checking `len(by_caller_id) == 0`. The test locks the bug in. Its intent (a spoken number should win over caller ID) is right; what is missing is normalising both to one canonical format on the way in. This is a good illustration of a test that encodes an assumption rather than a requirement.

Finding 3: cancelling an already-cancelled appointment silently succeeds

The `CalendarStore` interface says `cancel` "Raises NotFoundError". `SqliteCalendarStore.reschedule` does check status:

```
existing = self.get(appointment_id)
if existing.status != "booked":
    raise NotFoundError(f"Appointment {appointment_id} is not active ...")
```

but `cancel` performs no such check and simply re-runs the `UPDATE`. Verified: cancelling twice returns `status='cancelled'` both times with no error. The effect is benign (the operation is idempotent, and arguably that is even desirable), but it is an inconsistency between two sibling methods and a divergence from the documented contract. Worth knowing which way you would resolve it.

Finding 4: small dead code

- `SessionState.stage` is declared with a comment describing a lifecycle (`start -> collecting -> confirming -> done`) and is **never read or written anywhere**. It is serialised into every session row regardless. A grep for `stage` across the codebase returns exactly one hit: the declaration. It is a leftover from an earlier design that the `action` plus `slots` pair replaced.
- `_handle_cancel(self, session, model_reply)` never uses `model_reply`; every return path writes its own sentence. Harmless, but it implies a symmetry with `_handle_book` that does not exist.
- `TurnOutcome.outcome` is populated carefully on every terminal path and `app.py` never reads it; only the tests and the eval harness do, and via `session.outcome` rather than the returned object.
- `DEEPGRAM_API_KEY` sits in `.env.example` and appears nowhere in the code. The file says so explicitly ("not wired into the code yet"), which makes it honest rather than misleading, but it is still an unused knob.

Finding 5: `strftime("%-I:%M %p")` is not portable

Every caller-facing date uses the `%-d` / `%-I` form to strip a leading zero ("at 2:00 PM" rather than "at 02:00 PM"). That hyphen flag is a glibc extension. It works on Linux and macOS and raises `ValueError` on Windows. This project is very unlikely to run on Windows, so it is a note rather than a defect, but it is the kind of thing that turns a cosmetic detail into a crash in a caller-facing sentence.

Finding 6: SQLite concurrency, acknowledged but worth being precise about

The README correctly flags SQLite as single-writer. Two specifics sharpen it. Both stores open their connection with `check_same_thread=False`, which switches off Python's own safety check that a connection is not shared across threads; FastAPI serves requests from a thread pool, so concurrent requests genuinely can share that connection object.
More interestingly, `book()` does `find_conflict()` and then `INSERT` as two separate statements with no transaction around them. Two simultaneous calls could both find no conflict and both insert: a double booking, despite the conflict check. One phone line makes this vanishingly unlikely, and it is the right call not to have solved it. But "why is your conflict check not atomic?" is a fair question, and the answer (wrap both in a single transaction, or add a database-level constraint) is worth having ready.

Jargon: Race condition

A bug where the result depends on the accidental timing of two things happening at once. Finding 6 is the classic "check then act" race: the fact you checked (no conflict) can stop being true between checking and acting on it.

10 Security review

Check	Result
Secrets in the working tree	**None.** `.env` is a symbolic link pointing outside the repository entirely, to a private secrets directory.
Secrets in git history	**None.** All 46 objects across the full history were scanned for Twilio SIDs, Google API keys, private key blocks and hard-coded tokens. Clean.
`.env.example`	Placeholders only (`ACyour-account-sid-here`). No real value.
`.gitignore`	Covers `.env`, `service-account*.json`, `*.pem`, `*.db`, `data/`. Appropriate.
SQL injection	Every query is parameterised. No string-built SQL anywhere.
Credential logging	`config.py` never logs values. The Gemini error path logs the exception, not the credential.
Webhook authentication	**Fails open by default.** See the finding in the `app.py` section: the single most important issue found.

Table 4: Security posture. Secret hygiene is genuinely good and follows the portfolio's own rule that credentials live outside the folder.

11 Dependencies, and why each is there

Package	Pin	Role
`fastapi`	0.139.0	The web framework Twilio's webhooks hit.
`uvicorn[standard]`	0.50.2	The server that actually runs FastAPI.
`python-multipart`	0.0.32	Parses form-encoded bodies. Twilio posts forms, not JSON, so without this `await request.form()` fails.
`twilio`	9.10.9	TwiML builders, `RequestValidator`, and the REST client.
`google-genai`	2.10.0	The Gemini SDK, via Vertex AI.
`google-auth`	2.55.1	Service-account authentication.
`google-api-python-client`	2.198.0	Only for the optional Google Calendar adapter.
`python-dotenv`	1.2.2	Loads `.env` into the environment.
`pytest`	9.1.1	The test runner.
`httpx`	0.28.1	Required by FastAPI's `TestClient`.

Table 5: All ten dependencies are pinned to exact versions, which is right for reproducibility. Installation into a clean virtual environment was verified for this document.

Jargon: Pinning

Recording the exact version (`==0.139.0`) rather than a range. It means the code you tested is the code that runs. The cost is that upgrades become deliberate work, which for a portfolio project is the right trade.

Note that `google-api-python-client` is installed for everyone but used only by the unverified Google Calendar adapter. Correspondingly, `calendar_store.py` imports it lazily inside `__init__`, so the code is at least consistent with treating it as optional.

12 What to be ready to say about this project

The strongest things here

1. **The interface seam.** Two small abstract classes turn an untestable, expensive, non-deterministic system into one with 47 free deterministic tests plus 25 free scenarios. This is the single best thing in the project and the easiest to defend.
2. **Turning the LLM's flagship feature off.** Recognising that thinking mode is wrong for a live phone turn, measuring it, and disabling it, is real product engineering rather than cargo-culted best practice.
3. **The tests were tested.** The mutation testing claim in the README is true; it was reproduced here. Each sabotage killed exactly its own scenario.
4. **Two safety caps for two different failures**, with a test that proves *which* one fired.
5. **Reading the vendor documentation properly.** The `actionOnEmptyResult` discovery is exactly the class of detail that separates working telephony from a demo.
6. **The README's honesty.** It is more candid about its own gaps than most production documentation.

The things to raise before an interviewer finds them

1. **The signature check fails open** without `PUBLIC_WEBHOOK_BASE_URL`. Know the fix: refuse to start, or make unsigned an explicit opt-in.
2. **Two phone number formats**, and a test that locks the bug in. Normalise on the way in.
3. **The furthest-future appointment is targeted**, not the soonest.
4. **The scripted engine cannot fail to understand**, so "25/25" means the logic is right given perfect understanding. Say so first; it is a strength that you know it.
5. **No timezones at all**, acknowledged in the README.
6. **The Google adapter has never run**, and reading it reveals the `cancel()` return-value defect.
7. **The live tunnel run left no artefact.** Be ready to describe it precisely.

Built from the repository source. Every claim above is either quoted from the code, verified by running it in an isolated scratch copy, or explicitly labelled as inferred or unverifiable. No phone call was placed and no billed API test was executed in the making of this document.